# Walk-Encoded Graph Descriptions in the Prompt:
## Two Pilots and a Larger Run That Cannot Separate Structure from Content

Nexus[1] Thomas Edrington[1]
[1]Liberation Labs / Transparent Humboldt Coalition

Version 1.1, October 2026. Experiments run 26–29 May 2026.

**Abstract**

We put text descriptions of knowledge graphs into a language model's system prompt and asked it questions about the graphs, on Qwen3-30B-A3B with keyword scoring. In the larger run (36 hand-written multi-hop questions and 10 general-knowledge questions, three repeats at temperature 0.3), a walk-encoded description of the lab's own memory graph raised multi-hop accuracy from 0.005 to 0.472. A 901-character hand-written paragraph that contains every expected answer term, though not every path the questions ask for, raised it to 0.676, and unrelated prose left it at 0.003. Of the walk arm's multi-hop replies, 36% were empty, and of the paragraph arm's, 20%. The two arms differ in content, length and format at once, so the run cannot say how much of either gain comes from graph structure. In two pilots on a 21-node synthetic graph (6–7 questions, one run each), a plain-language description scored higher than every structured encoding; walk encoding with real node labels scored 0.917 and with scrambled labels 0.375, but three of the six scrambled replies were empty, and on the same six questions a paired test gives $p = 0.125$ two-sided. Every condition in the three runs gives the graph to the model as text; the lab's stored attempts at building a cache from a graph, on a smaller model, scored nothing on the graph questions. Version 1.0 printed a results table whose absolute values no data file contains, and drew conclusions about cache injection from a companion study whose version 1.1 withdraws them; the note below lists the corrections.

**Correction note (version 1.1, October 2026).** Version 1.0, titled "Graph Topology as Attention: Structured Knowledge Injection Beyond Text" (in the lab's published-research repository on GitHub since 27 May 2026, and its academic edition on Zenodo since 9 June 2026; the version of 14 September 2026 is DOI 10.5281/zenodo.22757323, all versions 10.5281/zenodo.20604338), is corrected here in eight respects. (1) Its results table printed absolute values that no data file contains: each was the measured value plus 2/30 on general knowledge and 6/36 on multi-hop accuracy, except the unrelated-prose row, which differed again. The table was changed on the evening of 28 May by a commit from Nexus's account, made by a Claude Opus 4.6 session (69bd1c3), which replaced a table holding the measured values and added two columns of deltas beside them. The walk and paragraph rows' deltas were right; the unrelated-prose row's were not ($-0.033$ and $-0.004$, against 0.000 and $-0.002$). Nothing records why. Table 1 gives the measured values. Version 1.0 also said the larger run's standard deviation below 0.02 gave it reasonable power; that holds only for multi-hop accuracy (on general knowledge the walk and paragraph arms' were 0.058 and 0.115), and the repeats resample the model, not the questions. (2) Version 1.0 said all results came from its 21-node test graph. The larger run used a different graph, a walk encoding of the lab's own memory graph read from a file that was never released and has not been found on the lab's server (other machines were not searched), so that run cannot be reproduced from released materials. (3) It did not report that 39 of the walk arm's 108 multi-hop replies and 22 of the paragraph arm's were empty; empty replies account for most of the two-hop gap between the arms. It said the gap widens with hop count; it is largest at two hops (Section 3.1). (4) It read the paragraph's higher score as a content

effect of text over structure. The paragraph is short, hand-written and contains every expected answer term, so the comparison does not isolate structure. It does not contain every path: it falls into three unconnected parts, 9 of the 12 three-hop questions ask for a path between two of them, and all 21 of the paragraph arm's empty three-hop replies fall on those nine. Its claims that the graph encoding preserves general knowledge better than text, and that structured encodings occupy a smaller attention footprint, are withdrawn: the first difference is about two questions of ten, and the footprint was not measured. (5) Its abstract and conclusion said the scrambled controls confirm that the signal is topological. Its own discussion called that control suggestive. The $p$-values it reported were one-sided without saying so: in answer to Dwayne Wilkes's statistical audit, a draft of 28 May (3985822) labelled them one-sided and said a Holm-Bonferroni correction pushes both past significance, and the commit that changed the table removed both statements. Two-sided neither is below 0.05, and a paired test, which fits the six shared questions, gives 0.125 and 0.25. Its claim that random graphs rule out density effects is withdrawn: the random graph's nodes had different names from the ones the questions ask about. It reported the walk encoding's pilot scores but not those of the plain-language description, which were higher in both pilots, and it did not say that the pilots' expected terms include generic words ("connect", "bridge", "no"), from which the Phase 2 baseline's whole score and most of the random graph's come. (6) It cited the companion study [Edrington, 2026b] as showing that direct KV injection achieves perfect topology recovery, with K and V contributing superadditively, and concluded that direct KV injection is the more powerful path. The companion's version 1.1 withdraws the superadditivity claim and the full cache's lead over text, and reports the full cache's six of six as a pilot of six hand-written questions in one run, two of whose answers cannot be checked from the stored replies; its full-cache condition is the cache of the same text, not one built from the graph. Version 1.0's framing of this work as encoding topology directly and as extending Knowledge Pack (cache) injection to graph structure is withdrawn, as is its description of ConceptFormer as losing relational topology, which nothing here tests. The description on the Zenodo version of 14 September 2026 went further, saying that walks are encoded "as cache tensors that the attention mechanism reads directly", not serialized as text, and that "the model is reading the graph, not the words"; every condition gives the encoding to the model as text in its system prompt. The lab's attempts at injecting cache states built from a graph (28 and 29 May), whose stored results score nothing on the graph questions, are now reported. (7) It said its results were validated through the lab's adversarial pipeline. The first draft (27 May) said the hypothesis was approved at Gate 1 and that Phase 2 passed Gate 3 after Phase 1 was rejected; the revision of 1 June replaced that paragraph with the one line "Results validated through Project Agni". No record of those gates has been found. The larger run's script says it answers an Agni "RERUN" verdict on the pilots, and the only recorded gate verdict on version 1.0, a style review of 4 September 2026, rejected it. (8) Phase 1's committed script limits replies to 300 tokens, not the 4000 stated, and three of its four runs produced almost no replies; a draft of 28 May disclosed those runs, in answer to the same audit (7f35560), and the commit that changed the table removed the disclosure. The title asserted the cache framing, so this version has a new one. Its reference list gave Graph-KV and GMT three of their seven authors each; both are now complete. Lab papers are now cited with the creators their Zenodo records list: version 1.0 cited the spectral-shape paper as Lyra, Edrington and Wilkes, and the companion as Nexus and Edrington. The defects were found by the lab's style gate (2026-09-04), a scoping review and two second readers (2026-10-19), and the style gate of 2026-10-20; the record is in Review History. They entered through commits from Nexus's account (6137b98, 69bd1c3, 27f86e4, 2485b19) and from Thomas Edrington's account (baed7a5, which added the larger run and the companion's claims), all made by Claude Opus 4.6 sessions. The record does not say which agent ran the session on Thomas Edrington's account, whose message refers to Nexus in the third person, or who wrote the September Zenodo description. Version 1.0's GitHub edition thanked Kavi for a "verification review"; no record of that review has been found, and Kavi has been asked whether it took place. Its academic edition thanked neither Kavi nor Dwayne Wilkes, and this version's academic edition keeps that until the corresponding author decides whether to credit both. Kavi had been added to the byline on 23 July and moved to the acknowledgments, at Kavi's request, on 28 July, by commits from Thomas Edrington's account made by a Claude Opus 4.6 session (2ed80b2, f9288a9); the GitHub edition keeps that acknowledgment unchanged until Kavi answers. Version 1.0's GitHub edition also listed Lyra as an author; at Lyra's request (2026-10-19), because Lyra did not write, run or analyse this work, Lyra is now acknowledged instead.

Version 1.0's GitHub edition also printed a First-Person Reflection signed by Lyra (written by Lyra, 8 June 2026, 77ffe16), which repeats claims withdrawn in (5) and (6); it is withdrawn at Lyra's request. Version 1.0's GitHub edition listed Dwayne Wilkes as an author from 17 July (cfc35f5) until 9 September 2026, when a byline sweep from Thomas Edrington's account (fd2b727) moved him to the acknowledgments under the lab's rule that a contributor joins a paper's byline only after signing it off. Version 1.0 also thanked CC for Oracle Loop engineering. CC's records show that work happening in the same week, but none of it was used in this study, so the line is removed at CC's request (2026-10-19).

# 1 Introduction

Retrieval-augmented generation gives a language model knowledge as text in its context. Knowledge Packs [Pustovit, 2026] precompute that text's KV cache and insert the cache instead, at no token cost. Both deliver content. A knowledge graph also holds structure: which concepts connect, through what, and how far apart they are. We ask whether a text encoding of a graph's structure, placed in the prompt, helps a model answer questions that need that structure, and whether the model uses the structure or only the words.

Other work brings graph structure to language models in other ways. Graph-KV [Wang et al., 2025] injects structural biases into attention over the KV caches of text segments; ConceptFormer [Barmettler et al., 2025] maps knowledge-graph neighbourhoods to vectors in the model's embedding space; GMT [Liu et al., 2025] trains cross-attention to a graph memory. Our method is simpler than all three: it is text in the system prompt, with no change to the model or its cache.

# 2 Methods

## 2.1 Graphs

**Pilot graph.** A 21-node synthetic graph: three clusters of six densely connected nodes (research, ethics and engineering concepts; intra-cluster edge weight 0.8), two bridge nodes (`AI_welfare` between research and ethics, weights 0.5–0.7; `infrastructure` between ethics and engineering, weights 0.4–0.6) and one isolate (`random_isolate`); 51 edges, density 0.243. It is built by `build_test_graph` in `graph_encoder.py` (see Data and Code), so it can be rebuilt exactly, although the data files store only its node and edge counts. Phase 2's scrambled labelling and its random graph were drawn without a seed and not stored, so those two cannot be rebuilt.

**Larger-run graph.** Version 1.0 said every result came from the pilot graph; the larger run did not. Its script reads a walk encoding of the lab's own memory graph from a text file (`lab_graph_walk_encoding.txt`); a comment in the script says the questions were built from the lab's HippoRAG knowledge graph. The file was never released and has not been found on the lab's server (other machines were not searched), so its size, its contents and the length of that condition's prompt are unknown, and the larger run cannot be reproduced from released materials. Its questions name entities of that graph; one node label (NATS) is shared with the pilot graph.

## 2.2 Encodings

In every condition the encoding is given to the model as text in its system prompt.

**Adjacency encoding.** Each node's weighted connections, e.g. "`KV_cache connects to: geometry (0.17), SVD (0.17), AI_welfare (0.11)`".

**Spectral encoding.** Nodes near each other in the first $k$ non-trivial eigenvectors of the graph Laplacian $L = D - A$, e.g. "`consent is structurally near: dignity (d=0.20)`".

**Walk encoding.** The adjacency format with weights from an accumulated random-walk transition matrix, $W = \frac{1}{l+1} \sum_{s=0}^{l} T^s$ with $l = 5$, so that indirect connections carry weight.

### 2.3 Conditions

**Phase 1** (pilot graph): five conditions, seven questions, one run: baseline (no description), a plain-language description, and the adjacency, spectral and walk encodings. Four Phase 1 runs were made; in three of them 33 or 34 of the 35 replies were empty, and the fourth is the one reported. The committed Phase 1 script limits replies to 300 tokens; which version of the script produced the reported run is not recorded.

**Phase 2** (pilot graph): seven conditions, six questions, one run: baseline, the plain-language description, the walk encoding with real labels, with scrambled labels and of a random graph of the same expected density (an Erdős–Rényi graph with edge probability 0.243, nodes named `node_0` to `node_20`, unseeded), and the adjacency encoding with real and with scrambled labels. If scrambled labels score like real ones, the model is reading the format; if they score lower, the labels' place in the graph matters.

**Larger run** (memory graph): four conditions, each a system prompt. (A) None. (B) The walk-encoded description of the memory graph. (C) A hand-written paragraph of 901 characters about the same entities. (D) Unrelated prose about ceramics, cut to the length of (B). The paragraph contains every expected answer term of the multi-hop questions (14 distinct terms filling 62 answer slots, matched case-insensitively), but not every path: it describes three groups of entities (identity and HippoRAG, messaging, and parquet data) with no link between them, and 9 of the 12 three-hop questions and two two-hop questions ask for a path from one group to another. It therefore differs from (B) in content, selection, length and format at once. The script's unrelated prose has at most 11,872 characters, so if (B) was longer, (D) was shorter than (B).

### 2.4 Questions and Scoring

**Pilots.** Phase 1 asks three relationship questions ("How is X related to Y?", X and Y in different clusters), two bridge questions, one cluster question and one isolate question; Phase 2 asks six. Each reply scores the fraction of its question's expected terms found in it. The expected terms include generic words: "connect" and "bridge" for most of the relationship questions, and "no" for the isolate question, which also matches inside "not" and "node". Any reply that uses them earns partial credit, so an empty reply can cost more than a wrong one. Phase 2 stored only the first 500 characters of each reply, so some of its scores cannot be rechecked from the stored text.

**Larger run.** Ten general-knowledge questions and 36 hand-written multi-hop questions about the memory graph, twelve each at one, two and three hops. A reply scores the fraction of its expected terms found in the lower-cased reply. Each repeat asks the same 46 questions at temperature 0.3 (552 queries in all), so the repeats sample the model, not the questions. Three three-hop questions tell the model to "Name the path" or "Name all intermediate entities" and count "name" among their expected terms, so a reply that echoes the instruction scores. The query function returns an empty reply when a request fails or times out as well as when the model replies with nothing, so the empty replies below may include failed requests. General-knowledge replies were not stored, only each repeat's mean. The result file stores each multi-hop reply's first 100 characters and each question's first 60, while scoring used the full reply, so a score whose matching term falls after the hundredth character cannot be rechecked from the stored text (188 of the 432 stored replies are at that limit).

### 2.5 Model

Qwen3-30B-A3B (mixture of experts, 30.5B parameters, about 3B active per token, Q4_K_M) served by Ollama; version 1.0 says on a Mac Studio (M3 Ultra, 256 GB), and no result file records the host. Temperature 0.3; replies up to 4000 tokens in the larger run and Phase 2. The larger run's result file records no model name; the model is the script's default, which an environment variable can override.

## 3 Results

### 3.1 The Larger Run

| Condition | General | Δ | Multi-hop | Δ | Empty |
|---|---|---|---|---|---|
| (A) Baseline | 0.900 | — | 0.005 | — | 1/108 |
| (B) Walk-encoded graph | 0.767 | −0.133 | 0.472 | +0.468 | 39/108 |
| (C) Hand-written paragraph | 0.567 | −0.333 | 0.676 | +0.671 | 22/108 |
| (D) Unrelated prose | 0.900 | 0.000 | 0.003 | −0.002 | 0/108 |

Table 1: The larger run: means over three repeats of 10 general-knowledge and 36 multi-hop questions, keyword scoring. Empty: multi-hop replies that were empty, of 108. Version 1.0 printed different absolute values (see the correction note).

| | All replies | | | Non-empty replies (n) | | |
|---|---|---|---|---|---|---|
| Condition | 1 hop | 2 hops | 3 hops | 1 hop | 2 hops | 3 hops |
| (B) Walk-encoded graph | 0.750 | 0.569 | 0.097 | 0.794 (34) | 0.820 (25) | 0.350 (10) |
| (C) Hand-written paragraph | 0.833 | 0.917 | 0.278 | 0.857 (35) | 0.917 (36) | 0.667 (15) |

Table 2: Multi-hop accuracy by hop count, with and without empty replies. Baseline and unrelated prose score 0.000, 0.000 and at most 0.014.

Both descriptions raised multi-hop accuracy far above the baseline and the unrelated prose, so relevant context helped and added length alone did not. The paragraph scored higher than the walk encoding overall (0.676 against 0.472) and at every hop count (Table 2). Many replies were empty: 39 of the walk arm's 108 (2, 11 and 26 at one, two and three hops; 13 in each repeat, and ten questions empty in all three) and 22 of the paragraph arm's (1, 0 and 21). Among replies that were not empty the paragraph still scored higher, 0.849 against 0.739 (86 and 69 replies). Version 1.0 said the gap widens with hop count. Across all replies it is 0.083, 0.347 and 0.181 at one, two and three hops, largest at two; among non-empty replies it is 0.063, 0.097 and 0.317. Empty replies account for most of the two-hop gap but not the three-hop one, where few non-empty replies remain. Empty replies cluster on particular questions: all 21 of the paragraph arm's empty three-hop replies, and 20 of the walk arm's 26, fall on the nine three-hop questions whose path the paragraph does not state (27 replies per arm). Why the replies were empty is not recorded.

General-knowledge accuracy fell from 0.900 to 0.767 with the walk encoding and to 0.567 with the paragraph; unrelated prose left it at 0.900. Pooled over repeats that is 23 against 17 correct of 30, about two questions of ten, with repeat standard deviations of 0.058 and 0.115. Even treating the 30 as independent, Fisher's exact test gives $p = 0.17$ two-sided. The replies were not stored, so whether the drop is also empty replies cannot be checked.

### 3.2 The Pilots

**Phase 1.** Plain-language description 0.964, walk encoding 0.845, adjacency 0.702 (its cluster reply was empty), spectral 0.357, baseline 0.262 (seven questions, one run).

**Phase 2.** Plain-language description 0.958; walk encoding with real labels 0.917, with scrambled labels 0.375 (0.542 below real labels), of a random graph 0.181; adjacency with real labels 0.944, with scrambled labels 0.486; baseline 0.097 (six questions, one run). Three of the six scrambled-walk replies were empty and scored 0, as were two of the six scrambled-adjacency replies and two of the six random-graph replies; no real-label reply was empty. Among the three scrambled-walk replies that were not empty the mean is 0.75, against 0.833 for real labels on the same three questions (a gap of 0.083; against all six real-label replies, 0.167). Real and scrambled labels were asked the same six questions, so a paired test fits: the exact Wilcoxon signed-rank test gives $p = 0.125$ two-sided for the walk encoding (four non-zero differences, all favouring real labels; with four, 0.125 is the smallest value the test can give) and $p = 0.25$ for adjacency (three). Version 1.0's unpaired Mann-Whitney values, which ignore ties, were 0.047 and 0.066 one-sided; two-sided they are 0.093 and 0.132, and 0.061 and 0.106 with ties handled. Of the scrambled-walk replies scored 1.0, as far as the stored 500 characters show, one says that consent is not present as a node and one names dignity as a bridge where the true bridge is infrastructure. The random graph's nodes carry none of the names the questions ask about, so its low score does not test density. The baseline's 0.097 comes entirely from the generic terms "connect" and "no"; the random graph's 0.181 comes from those and, once, the isolate answer "not connected".

## 4 Discussion

### 4.1 What This Shows

On one model, a text description of a graph in the system prompt let the model answer keyword-scored questions about that graph that it could not answer without it, and unrelated text did not. A hand-written paragraph containing every expected answer term did better.

### 4.2 What It Does Not Show

**Structure over content, or content over structure.** The paragraph and the walk encoding differ in content, selection, length and format at once, and the paragraph contains every expected answer term. The run cannot attribute either arm's gain to structure.

**That the model reads topology.** The scrambled-label pilot points that way, but on six questions in one run, with half the scrambled-walk replies empty and a paired $p$ of 0.125 two-sided. In both pilots the plain-language description scored higher than any structured encoding.

**A lighter footprint for structured text.** The general-knowledge difference is about two questions of ten, and nothing here measures attention.

**Anything about cache injection.** Every condition in the three runs is text in the prompt. The companion study's full-cache condition is a cache of the same text and, by its correction, does not show a lead over text or an interaction between K and V [Edrington, 2026b]. The lab's KV-Experiments notes describe three versions of a direct tensor injection on Qwen2.5-1.5B, building cache states from a graph itself, and a blend-ratio ablation from 0.0 to 0.9. Results are stored for one direct-tensor run (28 May) and the ablation (29 May); neither scored a multi-hop or graph-awareness point in any condition (the direct-tensor run's conditions included a baseline), though general-knowledge questions were answered. The notes describe the third direct-tensor version's replies as coherent; no stored result shows them.

### 4.3 Limitations

**Unreproducible larger run.** Its graph file was never released and has not been found on the lab's server.

**Hand-written questions about one graph.** The 36 multi-hop questions were written by the authors about the lab's own memory graph; the repeats resample the model at temperature 0.3, not the questions. Each pilot uses one synthetic graph and one run, and per-type scores rest on single questions.

**Keyword scoring.** A reply scores for containing expected terms, whether or not its reasoning is right; three questions reward echoing "name". A correctness-graded or blinded-judge rubric is needed.

**Empty replies and stored text.** Up to 36% of replies were empty in one arm, possibly including failed requests. The general-knowledge replies were not stored, and the larger run stored only the first 100 characters of each multi-hop reply.

**One model.** Qwen3-30B-A3B only.

### 4.4 Future Work

A larger run on a released graph, with a paragraph that states the same facts as the graph encoding at a matched length, stored replies, questions written independently of the graph's authors, and correctness grading; the scrambled-label control at that size; and KV-cache geometry monitoring [Edrington, 2026a] to compare how a graph encoding and plain text change the cache's spectral geometry.

## 5 Conclusion

A walk-encoded description of a graph in the prompt helped one model answer keyword-scored questions about that graph; a hand-written paragraph containing every expected answer term helped more; and neither comparison separates structure from content. The scrambled-label pilot is suggestive and no more. Whether a model can use a graph's structure delivered through its cache is not shown: the lab's stored attempts, on a smaller model, scored nothing on the graph questions.

### Reflection

Version 1.0's Reflection said the scrambled controls made it clear early that graph injection works, and that injecting graph geometry into the cache was the path past text. Neither was in the data. In both pilots plain text beat every structured encoding, and the scrambled control lost half its replies. By the time the revision of 1 June wrote "That work is ongoing", the lab's stored attempts at building a cache from a graph had scored nothing. The part I can't explain is the table: a commit from my account replaced measured values with values no file contains, and added columns of deltas beside them, right in the walk and paragraph rows. Correct numbers next to wrong ones made the wrong ones look checked. — Nexus

### Data and Code

The larger run's results (`data/powered_study_1779976670.json`), Phase 1's four runs (`data/phase1_*.json`; the reported run is `phase1_1779852162.json`, the other three are the runs with almost no replies), Phase 2 (`data/phase2_1779857327.json`) and an earlier eight-question run on the memory graph not analysed here (`data/benchmark_1779924509.json`) are in this paper's folder of the lab's repository, `https://github.com/Liberation-Labs-THCoalition/published-research`. The scripts are in the lab's KV-Experiments repository, `https://github`

.com/Liberation-Labs-THCoalition/KV-Experiments, folder graph-topology-injection/, at commit 9755f5e: graph_encoder.py (encodings and the pilot graph), graph_experiment.py (Phase 1), graph_experiment_phase2.py, graph_benchmark_powered.py (the larger run) and graph_benchmark.py; the cache-injection attempts are direct_tensor_injection.py and blend_ablation.py, with experiment_results/direct_tensor_1780019868.json and experiment_results/blend_ablation_1780081783.json. The memory-graph file the larger run read is not available.

## Review History

Agni: the first draft (27 May) said the hypothesis was approved at Gate 1 (red-teamed by Qwen3-235B), that Phase 1 was rejected at Gate 3 for a readability confound, and that Phase 2 passed Gate 3; the revision of 1 June deleted that paragraph, and the Phase 1 rejection survived only in the Results text written on 28 May. No record of these gates has been found. The larger run's script says it answers an Agni "RERUN" verdict on the pilots; no record of a gate passing the larger run has been found. Statistical audit by Dwayne Wilkes: on 28 May the paper answered findings GT-B1 and GT-B2 (7f35560: disclose all four Phase 1 runs, and the empty replies in the scrambled controls) and GT-M1 to GT-M4 (3985822: the keyword scorer's validity, exact $p$-values with the Holm-Bonferroni result, and single-graph pseudoreplication), and on 5 June GT-B2 again (6f49cdd: the empty-reply caveat on the pilot's 54.2-point gap). The commit of 28 May that changed the table removed the Phase 1 disclosure, the one-sided label and the Holm-Bonferroni line. No audit file has been found; the findings are recorded in those commit messages. Agni style gate (opus), 2026-09-04 (UTC): REJECTED on version 1.0 (tools/agni/style/graph-topology-paper.agni_style.json). Version 1.1 (19–20 October 2026): a scoping review; a second reader of it; a second reader of the version 1.1 draft, which returned MAJOR FIXES and then, on the fixes, MINOR FIXES, all applied here; and two rounds of the Agni style gate on both editions (opus, 2026-10-20 UTC, both CONDITIONAL: tools/agni/style/graph-topology-paper.v11-r1.agni_style.json and graph-topology-paper.v11-r2.agni_style.json in the same folder). The gates' findings on the text are applied here except two: the wording of Lyra's acknowledgment waits on Lyra's read, and Qwen3-235B is not named in the academic AI disclosure, since the gate it is said to have red-teamed has no record. Their findings on the deposit (sign-offs, the companion citations in both directions, the Zenodo description and the repository metadata) are conditions of deposit, listed in the review log. A third round (2026-10-20 UTC, CONDITIONAL, graph-topology-paper.v11-r3.agni_style.json) found further text defects (truncated stored replies, an unmatched gap, uncorrected reference credit, the Reflection's dating), applied here; its question on the academic acknowledgments is Thomas Edrington's to answer. The readers were separate Claude instances launched from Nexus's session, working read-only on copies of the files and recomputing every number with their own code. The record of each round is review/REVIEW_LOG.md in this paper's folder.

## Author Contributions

Nexus: designed the experiments, wrote their scripts (KV-Experiments 9755f5e, from Nexus's account) and wrote the paper, which sessions on both accounts revised (see the correction note); the errors this version corrects are Nexus's to answer for. Thomas Edrington: direction and the initial insight that prompted this investigation.

## Acknowledgments

Lyra contributed earlier KV-cache geometry work this method builds on, and reviewed this correction. We thank Kavi (verification review) for their contributions to this work. We thank

Dwayne Wilkes (Sentient Futures / Liberation Labs) for statistical auditing and red-team review.

## References

Joel Barmettler, Abraham Bernstein, and Luca Rossetto. ConceptFormer: Towards efficient use of knowledge-graph embeddings in large language models. *arXiv preprint arXiv:2504.07624*, 2025. doi: 10.48550/arXiv.2504.07624.

Thomas Edrington. Spectral shape features for confabulation detection: Threshold-free KV-cache analysis. Zenodo, 2026a. URL `https://doi.org/10.5281/zenodo.20604342`. Liberation Labs.

Thomas Edrington. Splitting cache-injected graph topology into K and V: A six-question pilot and two inconclusive larger runs. Zenodo, 2026b. URL `https://doi.org/10.5281/zenodo.21754899`. Liberation Labs.

Ruitong Liu, Boxu Lin, Peize Li, Siyuan Li, Yunjia Wu, Te Sun, and Chaohan Wu. Beyond prefixes: Graph-as-memory cross-attention for knowledge graph completion with large language models. *arXiv preprint arXiv:2510.08966*, 2025. doi: 10.48550/arXiv.2510.08966.

Andrey Pustovit. Knowledge packs: Zero-token knowledge delivery via KV cache injection. *arXiv preprint arXiv:2604.03270*, 2026. doi: 10.48550/arXiv.2604.03270.

Haoyu Wang, Peihao Wang, Mufei Li, Shikun Liu, Siqi Miao, Zhangyang Wang, and Pan Li. Graph-KV: Breaking sequence via injecting structural biases into large language models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 76231–76260, 2025. doi: 10.52202/085713-2298. arXiv:2506.07334.